Содержание

1 Задача 3.1.6d **1**

2 Псевдокод **1**

3 Решение **1**

4 Сложность **5**

1 Задача 3.1.6d

Условие: Отсортировать последовательность $a_1, \ldots, a_n$ в порядке возрастания элементов. Выбрана сортировка HeapSort.
in: $n, a_1, \ldots, a_n$
out: $b_1, \ldots, b_n$, где $b_1 \leq b_2 \leq \ldots \leq b_n$

2 Псевдокод

Псевдокод HeapSort, приведённый в исходном файле:

```
public static void sort(Comparable[] pq) {
    int n = pq.length;
    for (int k = n/2; k >= 1; k-)
        sink(pq, k, n);
    int k = n;
    while (k > 1) {
        exch(pq, 1, k-);
        sink(pq, 1, k);
    }
}

private static void sink(Comparable[] pq, int l, int m) {
    while (2*l <= m) {
        int j = 2*l;
        if (j < m && less(pq, j, j+1)) j++;
        if (!less(pq, l, j)) break;
        exch(pq, l, j);
        l = j;
    }
}
```

3 Решение

```
; Data Model
;0: r0
;1: trash | iterator
;2: &end
;3: n
;4: k
```

```
;5: l
;6: m
;7: j
;8: call (from what place was sink called last time)
;9 ... 9+n-1: arr

Load =0
Read 3
Sub 3           ; r3 := n
JZero END
JGTZ END  ; n <= 0 ==> exit (invalid size)

Load 3
Add =9
Store 2   ; r2 := 9 + n (arr_a.end())

Load =9
Store 1   ; r1 := 9 (arr_a.begin())

READ_ARRAY: Load 2
 Sub 1
 JZero SORT  ; if r2 == r1 exit read
 Read *1
 Load 1
 Add =1
 Store 1  ; r1++
 Jump READ_ARRAY
SORT: Load 3
    Div =2
    Store 4     ; r4 := k = n // 2
FOR: Load =1
    Sub 4
    JGTZ FOR_END    ; if k < 1 then exit for
    Load 4
    Store 5
    Load 3
    Store 6     ; l := k, m := n
    Load =0
    Store 8     ; call = 0 (call from for)
    Jump SINK   ; call sink(k, n)
FOR_CALL_END: Load 4
    Sub =1
    Store 4     ; k-
    Jump FOR
FOR_END: Load 3
    Store 4     ; k := n
WHILE_k_gt_1: Load =1
    Sub 4
    JZero PRINT_INIT
    JGTZ PRINT_INIT   ; if k <= 1 then exit while and exit sort
```

```
    ; SWAP
    Load 4
    Add =8
    Load *0
    Store 1     ; r1 := arr[k]
    Load 9      ; r0 := arr[1]
    Store 2     ; r2 := arr[1]
    Load 1
    Store 9     ; arr[1] := arr[k]
    Load 4
    Add =8
    Store 1     ; r1 := &arr[k]
    Load 2
    Store *1    ; arr[k] (= *r1) := arr[1] (= r2)
    ; k-
    Load 4
    Sub =1
    Store 4
    ; CALL sink(1, k)
    Load =1
    Store 5     ; l := 1
    Load 4
    Store 6     ; m := k
    Load =1
    Store 8     ; call := 1 (call sink from while)
    Jump SINK
SINK: Load =2
    Mult 5
    Sub 6
    JGTZ CALL_RETURN    ; if 2*l-m > 0 then exit SINK
    Load =2
    Mult 5
    Store 7     ; j := 2*l
    ; if (j < m && less(pq, j, j+1)) j++;
    Load 7
    Sub 6
    JZero IF_BREAK
    JGTZ IF_BREAK       ; if j-m>=0 then do not do IF
    Load 7
    Add =8
    Store 1     ; r1 := &arr[j]
    Add =1
    Store 2     ; r2 := &arr[j+1]
    Load *1
    Sub *2
    JZero IF_BREAK
    JGTZ IF_BREAK       ; if a[j] >= a[j+1] then do not do IF
    Load 7
    Add =1
    Store 7     ; j++
```

```
; if (!less(pq, l, j)) break;
IF_BREAK: Load 7
    Add =8
    Load *0
    Store 1     ; r1 := arr[j]
    Load 5
    Add =8
    Load *0     ; r0 := arr[l]
    Sub 1       ; r0 := arr[l] - arr[j]
    JZero CALL_RETURN
    JGTZ CALL_RETURN    ; if a[l] >= a[j] then return from SINK
    ; SWAP if not break executed
    Load 7
    Add =8
    Load *0
    Store 1     ; r1 := arr[j]
    Load 7
    Add =8      ; r0 := &arr[j]
    Store 2     ; r2 := &arr[j]
    Load 5
    Add =8
    Load *0     ; r0 := arr[l]
    Store *2    ; arr[j] (= *r2) := arr[l] (= r0)
    Load 5
    Add =8
    Store 2     ; r2 := &arr[l]
    Load 1      ; r0 := arr[j] (previous)
    Store *2    ; arr[l] (= *r2) := arr_previous[j] (= r0)
    ; l = j
    Load 7
    Store 5
    ; goto WHILE
    Jump SINK
CALL_RETURN: Load 8
    JZero FOR_CALL_END  ; return from sink(k, n)
    Jump WHILE_k_gt_1   ; return from sink(1, k)
PRINT_INIT: Load =9
    Store 1     ; r1 := arr.begin()
    Add 3
    Store 2     ; r2 := arr.end()
PRINT: Load 2
    Sub 1
    JZero END   ; if r1 == r2 then exit print
    Write *1
    Load 1
    Add =1
    Store 1     ; r1++
    Jump PRINT
END: Halt 0
```

4 Сложность

Пусть $h := \lfloor n/2 \rfloor$.

До вызовов $SINK$:

1. Проверка входа - 5 шагов
2. Инициализация указателей - 5 шагов
3. Чтение массива $READ_ARRAY$ - n итераций:
   - 3 шага на проверку конца массива
   - 5 шагов на тело цикла

   Последняя проверка конца массива - 3 шага. Итого: $8n + 3$.
4. Инициализация $SORT$ до FOR - 3 шага
5. Построение кучи, цикл FOR без вызовов $SINK$: h итераций:
   - проверка FOR - 3 шага
   - подготовка вызова $SINK$ - 7 шагов
   - FOR_CALL_END - 4 шага

   Последняя проверка FOR - 3 шага; инициализация $k = n$ в FOR_END - 2 шага. Итого: $14h + 5$.
6. Извлечение максимума, цикл $WHILE_k_gt_1$ без вызовов $SINK$: $n - 1$ итераций:
   - проверка условия - 4 шага
   - обмен элементов - 13 шагов
   - уменьшение k - 3 шага
   - подготовка вызова $SINK$ - 7 шагов

   Последняя проверка цикла - 3 шага. Итого: $27(n - 1) + 3 = 27n - 24$.
7. $PRINT_INIT$, вывод массива $PRINT$ и завершение: 4 шага на инициализацию, n итераций вывода по 8 шагов, последняя проверка - 3 шага, $Halt$ - 1 шаг. Итого: $8n + 8$.

Следовательно часть без $SINK$:

$$T_{fixed}(n) = 5 + 5 + (8n + 3) + 3 + (14h + 5) + (27n - 24) + (8n + 8) = 43n + 14 \left\lfloor \frac{n}{2} \right\rfloor + 5$$

Подсчёт одного вызова $SINK$. Пусть $C_r(A, l, m)$ - число шагов от входа в $SINK$ до возврата к вызывающему циклу. $r = 0$ для вызова из FOR и $r = 1$ для вызова из $WHILE_k_gt_1$.

1. Проверка в начале $SINK$ - 4 шага. Если $2l > m$, выполняется $CALL_RETURN$ за $2 + r$ шага, поэтому
$$C_r(A, l, m) = 6 + r$$

2. Иначе $j := 2l$ - 3 шага. Пусть q - выбранный индекс, а B - число шагов этого блока:

$$(q, B) = \begin{cases} (j, 3), & j = m \\ (j, 4+8), & j < m,\ A_j = A_{j+1} \\ (j, 4+9), & j < m,\ A_j > A_{j+1} \\ (j+1, 4+12), & j < m,\ A_j < A_{j+1} \end{cases}$$

На сравнение $A_j = A_{j+1}$ - 8 шагов, $A_j > A_{j+1}$ - 9 шагов и $A_j < A_{j+1}$ - 12 шагов.

3. В IF_BREAK сравнение A_l и A_q - p шагов, где $p = 9$ при $A_l = A_q$ и $p = 10$ при $A_l > A_q$. Если $A_l \geq A_q$ - возврат за $2 + r$ шага. Если $A_l < A_q$, сравнение - 10 шагов, обмен и переход обратно к $SINK$ - 19 шагов.

Пусть A' - состояние массива после обмена элементов с индексами l и q. Тогда время одного вызова:

$$C_r(A, l, m) = \begin{cases} 6 + r, & 2l > m, \\ 7 + B + p + 2 + r, & 2l \leq m,\ A_l \geq A_q, \\ 7 + B + 10 + 19 + C_r(A', q, m), & 2l \leq m,\ A_l < A_q. \end{cases}$$

Значит, общее число шагов:

$$T(n, A) = T_{fixed}(n) + \sum_{k=1}^{h} C_0(A^{(k)}, k, n) + \sum_{m=1}^{n-1} C_1(A^{[m]}, 1, m)$$

где $A^{(k)}$ и $A^{[m]}$ - массив перед соответствующими вызовами $SINK$. Значит, число шагов в худшем случае:

$$T_{\max}(n) = \max_A T(n, A)$$

Переходы вниз. Переход вниз в SINK: после сравнения в блоке IF_BREAK: обмен, $l := j$, начало следующей итерации, то есть от метки SINK до Jump SINK.

1. От SINK до $j := 2l$ - 7 шагов

2. Выбор большего потомка: Если $j = m$, правого потомка нет и переход. Если $j < m$, проверка наличия правого потомка - 4 шага. Далее сравнение $a[j]$ и $a[j+1]$ - 8 шагов при равенстве, 9 при $a[j] > a[j+1]$ и 12 при $a[j] < a[j+1]$ и плюс последние 3 шага. Максимум - 16 шагов.

3. От IF_BREAK до CALL_RETURN сравнение $a[l]$ с выбранным потомком - не более 10 шагов

4. Если $a[l] < a[j]$ - обмен - 19 шагов

Значит один переход вниз занимает не более $7 + 16 + 10 + 19 = 52$ шагов.

Если переход вниз не выполняется, вызов заканчивается. Максимальный путь - IF_BREAK и CALL_RETURN: $7 + 16 + 10 + 3 = 36$ шагов. Для вызова из FOR - возврат 2 шага. Если нет левого потомка, входная проверка и возврат - $6 + r \leq 7$ шагов. Значит, завершающая часть любого вызова SINK - не более 36 шагов.

Число переходов вниз при построении кучи. В FOR $k \in h, h-1, \ldots, 1$, $h = \lfloor n/2 \rfloor$. Перед вызов SINK - $l := k$, $m := n$. После каждого обмена в SINK выбранный индекс потомка - $j = 2l$ или $j = 2l+1$, а потом $l := j$. Значит, после t переходов вниз индекс не меньше $k2^t$ и не больше n. Поэтому число переходов для вызова с начальным $l = k$ не больше $\lfloor \log_2(n/k) \rfloor$. Значит

$$H(n) = \sum_{k=1}^{h} \left\lfloor \log_2 \frac{n}{k} \right\rfloor$$

По формуле из статьи. Paparrizos, *A Tight Bound on the Worst-Case Number of Comparisons for Floyd's Heap Construction Algorithm*, §3, формула (3):

$$t(n) = n - \lceil \log_2(n+1) \rceil + \lambda(n),$$

где $t(n)$ - сумма высот поддеревьев, корни которых находятся во внутренних вершинах, а $\lambda(n)$ - число нулей в двоичной записи n. Для вершины с индексом k в массивном представлении кучи высота соответствующего поддерева равна $\lfloor \log_2(n/k) \rfloor$, поэтому

$$H(n) = n - \lceil \log_2(n+1) \rceil + \lambda(n) = n - s_2(n)$$

где $s_2(n)$ - число единиц в двоичной записи n. Тогда, время на все переходы вниз при построении кучи $\leq 52H(n)$, а завершающие части h вызовов - $36h$.

Число переходов вниз при извлечении максимума. В цикле WHILE_k_gt_1 сначала уменьшается k, потом $l := 1$, $m := k$ и SINK. Поэтому размеры кучи в этих вызовах: $m = n-1, n-2, \ldots, 1$. При размере m начальный индекс $l = 1$. После каждого перехода вниз индекс не меньше удвоенного предыдущего. Следовательно, число переходов не больше $\lfloor \log_2 m \rfloor$, и в сумме

$$D(n) = \sum_{m=1}^{n-1} \lfloor \log_2 m \rfloor$$

D. E. Knuth, *The Art of Computer Programming*, Vol. 1, §1.2.4, упражнение 42(b):

$$\sum_{k=1}^{N} \lfloor \log_b k \rfloor = (N+1)\lfloor \log_b N \rfloor - \frac{b^{\lfloor \log_b N \rfloor + 1} - b}{b-1}$$

$b = 2$, $N = n-1$, $L = \lfloor \log_2(n-1) \rfloor$:

$$D(n) = nL - \frac{2^{L+1} - 2}{1} = nL - 2^{L+1} + 2$$

Значит, время на все переходы вниз при извлечении $\leq 52D(n)$, а завершающие части $n-1$ вызовов - $36(n-1)$.

Всего вызовов SINK - $h+n-1$: h при построении кучи и $n-1$ при извлечении максимума. В итоге:

$$\begin{aligned} T_{\max}(n) &\leq T_{fixed}(n) + 52\big(H(n) + D(n)\big) + 36(h + n - 1) \\ &= 43n + 14\left\lfloor \frac{n}{2} \right\rfloor + 5 + 52\big(H(n) + D(n)\big) + 36\left(\left\lfloor \frac{n}{2} \right\rfloor + n - 1\right) \\ &= 43n + 14\left\lfloor \frac{n}{2} \right\rfloor + 5 + 52\left(n - s_2(n) + nL - 2^{L+1} + 2\right) + 36\left(\left\lfloor \frac{n}{2} \right\rfloor + n - 1\right) \end{aligned}$$

, $L = \lfloor \log_2(n-1) \rfloor$. Значит, в худшем $T(n) = O(n \log n)$.